MINISTRY OF EDUCATION AND TRAINING

FPT UNIVERSITY

Fraud Detection and Risk Scoring System Using Apache Spark and Machine Learning

by

RocketUpTeam

AN Duong Binh	TUYEN Le Quang
LONG Pham Duc	TRUNG Do Quoc
QUAN Duong Hong	NHI Nguyen Le Hong

Final Group Project Report submitted for BDA501 – Big Data

MINISTRY OF EDUCATION AND TRAINING

FPT UNIVERSITY

Fraud Detection and Risk Scoring System Using Apache Spark and Machine Learning

by

RocketUpTeam

AN Duong Binh	TUYEN Le Quang
LONG Pham Duc	TRUNG Do Quoc
QUAN Duong Hong	NHI Nguyen Le Hong

Final Group Project Report submitted for BDA501 – Big Data

Supervisor:

TAN Le Duy

Fraud Detection and Risk Scoring System Using Apache Spark and Machine Learning

RocketUpTeam

AN Duong Binh
TUYEN Le Quang
LONG Pham Duc
TRUNG Do Quoc
QUAN Duong Hong
NHI Nguyen Le Hong

Degree Master of Software Engineering

FPT University

2026

Abstract

This thesis presents an academic fraud risk scoring workflow for the IEEE-CIS dataset. Apache Spark performs typed ingestion, transaction–identity joining, quality auditing, chronological leakage-controlled processing, feature engineering, and Parquet handover. Python model training compares Logistic Regression, LightGBM, XGBoost, and CatBoost after the Spark-to-pandas boundary. The system packages feature mappings, calibration, thresholds, and explanations for an in-process FastAPI service with a React review interface.

The current CatBoost balanced candidate reports validation ROC-AUC 0.9081210 and PR-AUC 0.5805993, with holdout ROC-AUC 0.8780755 and PR-AUC 0.4266435. The current CatBoost candidate is labelled `v2` in its metadata, but its promotion decision is `not_promoted`; the configured V2 software default remains unchanged. The principal limitations are offline–online historical-feature mismatch, duplicated policy ownership, local lineage, single-node training, fail-open fallback, and incomplete security and monitoring controls. The result is a bounded academic demonstration, not a production-readiness claim.

ACKNOWLEDGMENTS

The members of RocketUpTeam sincerely thank supervisor TAN Le Duy for guidance and feedback provided throughout project development and documentation.

Team Roster and Repository Role Mapping

Table 1 lists the project team members, official roles, and primary repository responsibilities.

Table 1: *Team Roster and Repository Role Mapping*

Member Name	Role	Primary Responsibility
TUYEN Le Quang	Project Leader	Responsible for system ideation and overall project management.
NHI Nguyen Le Hong	Content & Presentation Coordinator	Responsible for synthesizing project content, designing slides, and developing the presentation strategy.
AN Duong Binh	Big Data & Pipeline Engineer	Responsible for Spark data ingestion, left joins, temporal splitting, and feature contract verification.
LONG Pham Duc	Machine Learning Engineer	Responsible for classifier training, hyperparameter tuning, isotonic calibration, and metric evaluation.
TRUNG Do Quoc	Backend & API Engineer	Responsible for FastAPI scoring service, database persistence, and scoring contracts.
QUAN Duong Hong	Frontend & UI Engineer	Responsible for React dashboard UI, risk visualizations, and analyst review workflow.

LIST OF ABBREVIATIONS AND ACRONYMS

Abbreviation	Full Definition / Meaning
API	Application Programming Interface
AUC	Area Under the Curve
BDA	Big Data Analytics
CSV	Comma-Separated Values
EDA	Exploratory Data Analysis
FPT	Corporation / FPT University
GBDT	Gradient Boosted Decision Trees
HTTP	Hypertext Transfer Protocol
IEEE-CIS	IEEE Computational Intelligence Society
JSON	JavaScript Object Notation
JVM	Java Virtual Machine
LR	Logistic Regression
ML	Machine Learning
MLlib	Machine Learning Library (Apache Spark)
PR-AUC	Precision-Recall Area Under the Curve
ROC-AUC	Receiver Operating Characteristic Area Under the Curve
SHAP	SHapley Additive exPlanations
SQL	Structured Query Language
UI	User Interface
V2	Version 2 Model Release Authority

CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1

INTRODUCTION

1.1 Problem context

Online transaction fraud is an imbalanced classification problem with rare positive instances and asymmetric error costs [1, 2]. A production fraud scoring system requires reproducible data pipelines, temporal validation integrity, explainable predictions, and an operational human-review workflow.

> How can a reproducible big-data pipeline, an imbalanced-classification model, an explainable scoring API, and a human-review interface be integrated into one auditable fraud risk scoring system?

1.2 Objectives & research questions

Core Objectives:

- Ingest and validate the **1.29 GB** IEEE-CIS transaction and identity dataset via Apache Spark.
- Construct leakage-free chronological dataset splits (`train`, `val`, `holdout`).
- Evaluate MLlib DecisionTree, LightGBM, XGBoost, and CatBoost classifiers.
- Serve packaged models behind a FastAPI scoring API with TreeSHAP explanations and React UI.

Research Questions:

RQ1: Can raw data be processed without identifier overlap or temporal leakage?
RQ2: Which classifier achieves highest validation PR-AUC and retains holdout performance?
RQ3: Can the model be served efficiently without a Spark dependency at request time?
RQ4: What technical boundaries separate the academic demo from production readiness?

1.3 Evaluation basis

Primary metrics are ROC-AUC and Precision-Recall AUC (PR-AUC). PR-AUC is authoritative for rare positive classes [3, 4].

1.4 BDA501 requirement compliance

Table 1.1 summarizes compliance with BDA501 project rubric requirements.

Table 1.1: *BDA501 Requirement-Compliance Matrix*

Requirement	Project evidence
Main tool: Apache Spark	PySpark DataFrame & MLlib pipeline in Spark Preprocessing Module
Dataset size $\geq$ **500 MB**	**4** raw IEEE-CIS CSV files totaling **1.29 GB** in raw dataset store
Raw-data ingestion	Explicit Spark schemas & required file validation in ingestion pipeline
Cleaning & transform	Stateless cleanup, identity normalization, missingness handling
Descriptive analysis	Spark `groupBy`, `agg`, `Window` analytics & EDA reporting module
Machine learning	PySpark MLlib DecisionTree baseline & CatBoost/LightGBM/XGBoost comparison
Distributed cluster	**2**-worker Docker Compose Spark standalone cluster configuration
Architecture diagram	End-to-end system architecture & contract diagrams in documentation
Installation instructions	Project `README` and Docker Compose full-stack orchestration
Main source code	Modular source tree (`pipeline`, `model`, `backend`, `frontend`)
Analysis results	Saved JSON/Parquet metrics, PR-AUC, confusion matrix, calibration reports
Challenges & solutions	Summarized in Section 1.4 & Technical Challenges Matrix
Demo readiness	Full-stack Docker Compose deployment with React UI & FastAPI scoring backend

CHAPTER 2

BACKGROUND AND RELATED WORK

2.1 Imbalanced temporal ranking

Fraud detection is framed as binary temporal ranking on rare positive cases (fraud rate $\approx$ **3.5%**). Chronological splitting along `TransactionDT` preserves temporal order and prevents lookahead leakage.

2.2 IEEE-CIS dataset structure

The dataset contains **590,540** training transactions and optional identity records [1]. Preserving transaction row authority via LEFT JOIN prevents data loss while capturing missingness indicators.

2.3 Technology boundary

Architecture operates via **distributed Spark data preparation with single-node Python estimator training**. CatBoost, LightGBM, XGBoost, and Logistic Regression are evaluated, using TreeSHAP for feature attribution [5].

CHAPTER 3

REQUIREMENTS AND SYSTEM ARCHITECTURE

3.1 Functional requirements

Table 3.1: *Functional requirements specification*

ID	Requirement Description
FR-01	Validate and process required IEEE-CIS transaction and identity CSV files.
FR-02	Produce train, validation, holdout, and unlabeled model-ready datasets.
FR-03	Preserve canonical feature order and preprocessing metadata.
FR-04	Compare binary classifiers using validation PR-AUC metrics.
FR-05	Score transactions and return probability, risk score, version, and explanations.
FR-06	Persist scored transactions and human review outcomes.
FR-07	Filter, sort, paginate, and inspect transaction records.
FR-08	Import CSV and load processed datasets with progress reporting.
FR-09	Support review approval/rejection and fraud/legitimate labels.
FR-10	Present risk, SHAP evidence, and model state in a browser interface.

3.2 Quality requirements

Table 3.2: *Quality requirements specification*

ID	Requirement Description
QR-01	Temporal leakage control and non-overlapping chronological splits.
QR-02	Reproducible schema and feature contracts.
QR-03	Version-aware model artifacts and rollback capability.
QR-04	Explainable tree-model predictions via TreeSHAP.
QR-05	Local containerized deployment with bounded data mounts.
QR-06	Automated subsystem test suite.
QR-07	Auditable experiment and promotion lineage.
QR-08	Durable background job operations.
QR-09	Role-based authentication, authorization, and audit logging.
QR-10	Real-time production latency and drift monitoring.

3.3 Integrated architecture

The system integrates batch data preparation, offline ML training, and online inference into an in-process modular monolith. Figure 3.1 presents the overall architecture and data flow contracts.

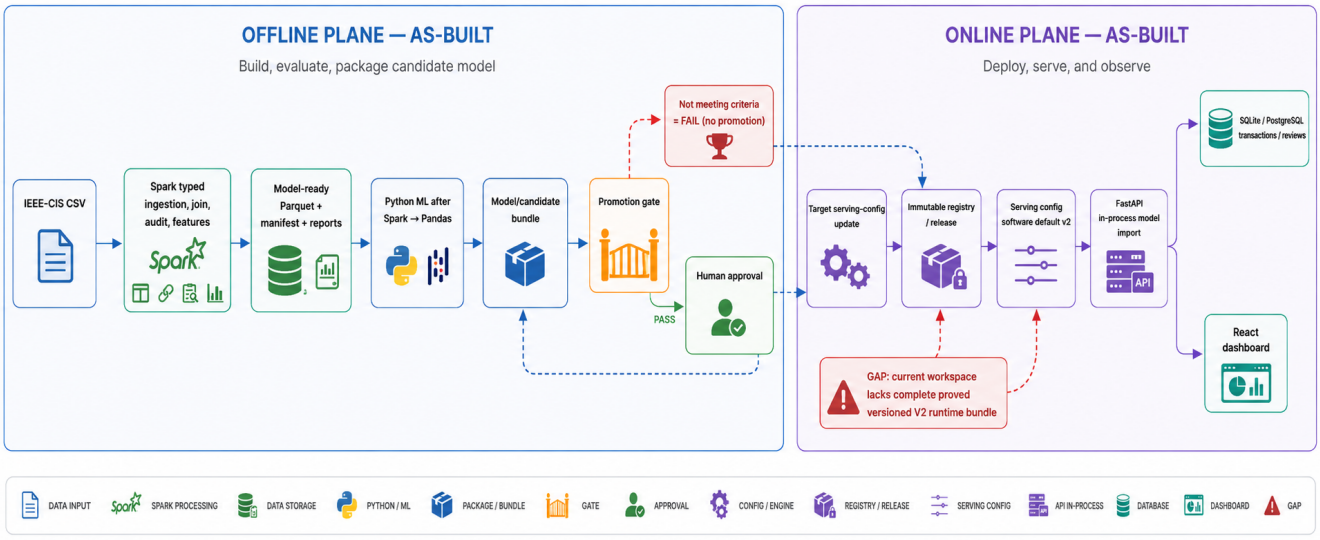


Figure 3.1: *Overall system architecture. Solid lines are implemented; dashed lines are proposed or partially enforced. The failed-candidate path leaves serving unchanged.*

3.4 System contracts

The system enforces three primary contracts:

1. **Data Contract**: Canonical 68-feature vector and schema hash.
2. **Scoring Contract**: Per-transaction request/response, probability, and TreeSHAP.
3. **API Contract**: Pydantic & TypeScript REST endpoints.

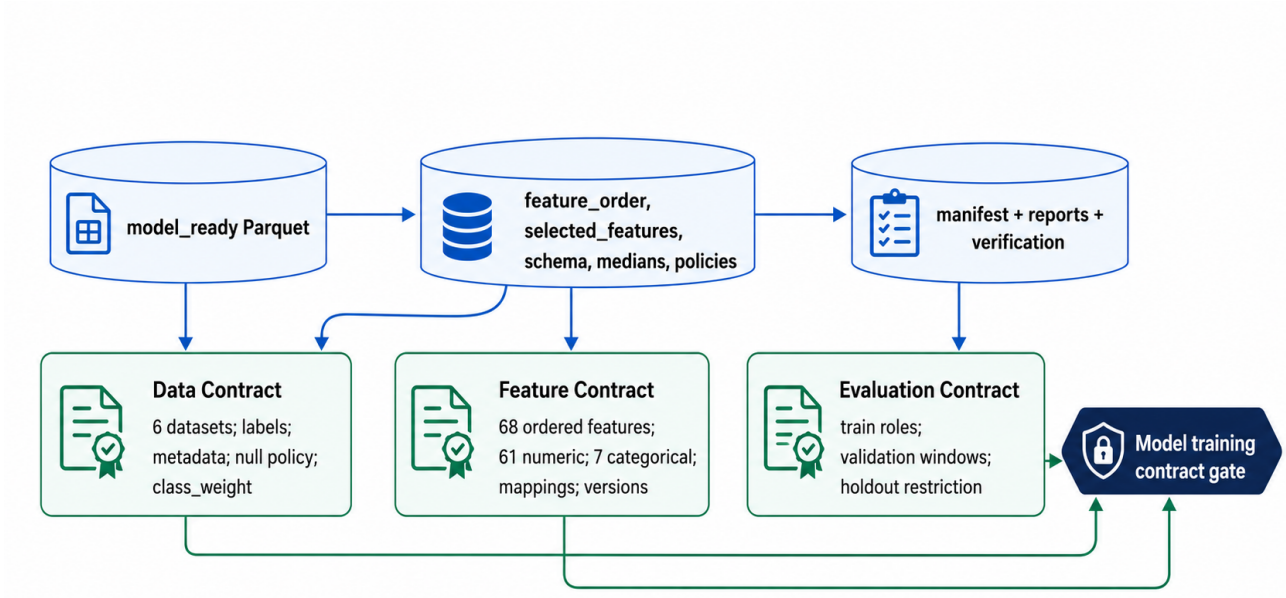


Figure 3.2: *Data, feature, and evaluation contracts. Split configuration and training protocol define the Evaluation Contract.*

3.5 Architectural tradeoffs

Table 3.3: *Major architectural tradeoffs*

Choice	Benefit	Cost
Monorepo and path dependency	Simple handoff and one Compose stack	Backend inherits model dependencies
Spark then pandas	Scalable preparation and familiar estimators	Driver-memory training ceiling
Fixed backend risk bands	Stable UI and simple tests	Not tied to model operating evidence
In-memory jobs	Minimal infrastructure	Lost on restart and single-worker only
SQLite local default	Fast developer startup	Different operational behavior from PostgreSQL
Heuristic fallback	Demo continuity	Can conceal model loading failures
Manual TypeScript types	No generator setup	Contract drift risk

3.6 Main source code components

Table 3.4 outlines the primary source implementation files and core responsibilities across project components.

Table 3.4: *Main Source Files and Component Responsibilities*

Component	Module reference	Responsibility
Spark Preprocessing	Preprocessing Pipeline	Schema ingestion, left join, leakage-free transformation, Parquet export
Data Verifier	Verification Engine	Validates dataset row counts, schema hash, split overlap, verification report
Model Training	Estimator Training Module	Single-node candidate training, tuning, calibration, holdout evaluation
Backend Scoring	FastAPI Scoring Service	Artifact loading, probability scoring, SHAP calculation, FastAPI endpoints
Frontend Dashboard	React UI Dashboard	React UI, risk queue, SHAP feature rendering, analyst review workflow
Docker Deployment	Multi-container Orchestration	Multi-container service orchestration (backend, frontend, db, preprocessing)

3.7 Distributed Spark execution evidence

Table 3.5 presents the Spark execution configurations and deployment specification available in the repository.

Table 3.5: *Distributed Spark Execution Evidence*

Spark mode	Master URL	Wk	Cpu	Mem	Out	Deployment Spec
Local Preproc	`local[4]`	**1**	**4**	**8GB**	**280MB**	Preprocessing Service Spec
Cluster Mode	`spark://master:7077`	**2**	**4**	**8GB**	**280MB**	Standalone Cluster Spec

Note: Cluster configuration is implemented, but a clean end-to-end distributed runtime was not independently verified.

CHAPTER 4

BIG-DATA PROCESSING AND FEATURE ENGINEERING

4.1 Source validation and typed ingestion

The raw inventory contains four required IEEE-CIS files totaling **1.29 GB**. The pipeline validates discovery, file names, sizes, headers, and schemas prior to Spark execution.

Table 4.1: *Observed source files*

File	Bytes	Role
`train_transaction.csv`	**683,351,067**	Labeled transactions
`train_identity.csv`	**26,529,680**	Optional train identity
`test_transaction.csv`	**613,194,934**	Unlabeled transactions
`test_identity.csv`	**25,797,161**	Optional test identity

Explicit Spark schemas prevent type inference discrepancies and ensure measurable conversion failures. Identity column names are normalized prior to joining.

4.2 Join integrity

The transaction table serves as row authority, joined with identity via a left join on `TransactionID`. Table 4.2 confirms row count preservation.

Table 4.2: *Transaction–identity join audit*

Dataset	Transactions	Identity matches	Unmatched	Row difference
Train	**590,540**	**144,233**	**446,307**	**0**
Test	**506,691**	**141,907**	**364,784**	**0**

Missing identity records are expected; presence flags preserve information without row loss.

4.3 Exploratory analysis and quality controls

The pipeline generates source inventory, key audits, duplicate records, missingness profiles, class distributions, and outlier reports. These reports are strictly descriptive and non-causal.

4.4 Leakage-controlled transformation order

To prevent data leakage, all learned transformations and aggregations fit strictly on training partitions and apply forward to validation and holdout sets. Figure 4.1 depicts stage-by-stage

transformations and output contracts.

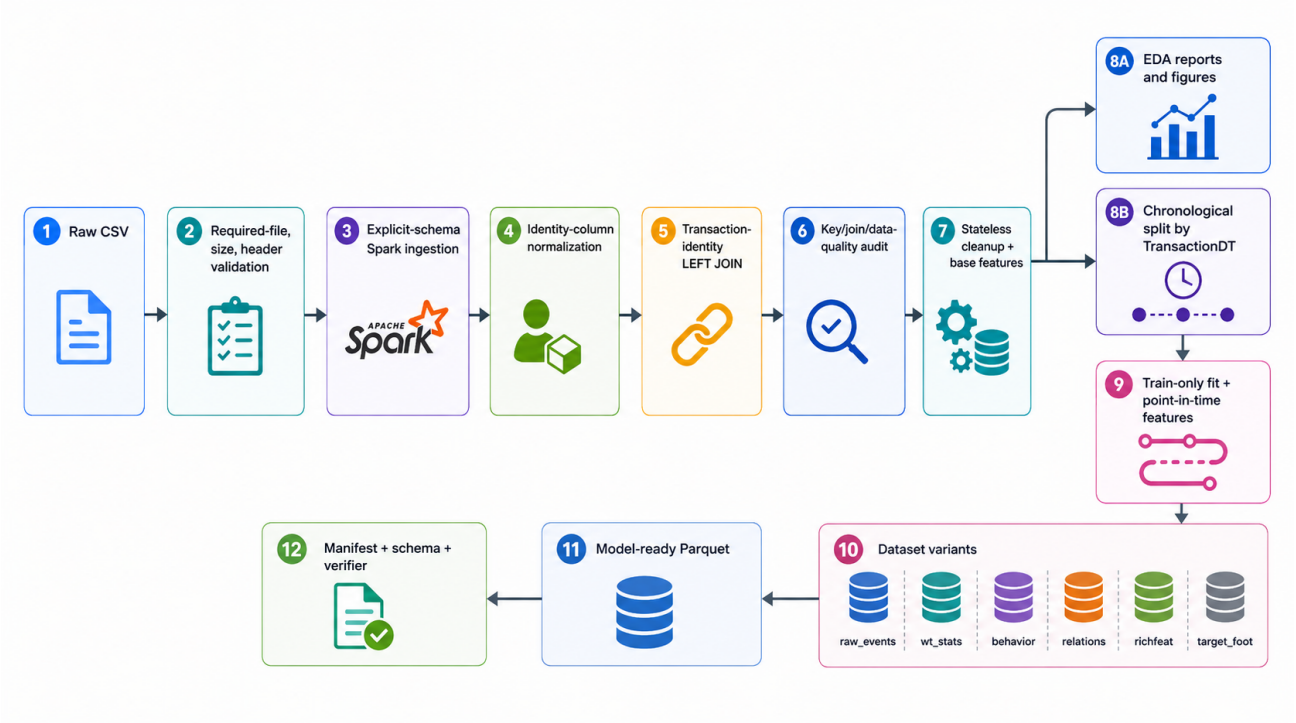


Figure 4.1: *Detailed data processing pipeline. Generates six canonical dataset partitions:* `train_original`, `train_weighted`, `train_balanced`, `validation`, `holdout`, *and* `kaggle_test`.

Sorting by `TransactionDT` creates forward temporal validation, reflecting real-world chronological scoring.

4.5 Chronological split

Table 4.3: *Chronological labeled splits*

Dataset	Rows	Fraud	Fraud rate	`TransactionDT` range
Train	412,956	14,522	3.5166%	86,400–10,432,902
Validation	88,486	3,036	3.4310%	10,432,915–13,136,583
Holdout	89,098	3,105	3.4849%	13,136,627–15,811,131

The verifier confirms zero identifier overlap between splits and stable fraud prevalence across chronological windows.

4.6 Feature engineering

4.6.1 Time features

Time features capture transaction day, week, hour, day-of-week, age, and night flags. Raw `TransactionDT` is retained in canonical feature order.

4.6.2 Amount features

Amount features include raw value, logarithm, capped value, decimal part, amount band, and card history deviations. Outlier thresholds fit on training only.

4.6.3 Missingness and presence

Missingness counts/ratios and presence flags cover identity, device, email, distance, address, and domain presence.

4.6.4 Anonymized behavior fields

Selected `C1--C14`, `D1--D15`, and distance attributes are retained as anonymized behavioral indicators.

4.6.5 Historical entity features

Card, email, and device keys provide prior transaction counts, amounts, standard deviations, and recency, fitted strictly from training.

4.6.6 Categorical features

Seven categorical fields (`ProductCD`, `card4`, `card6`, `DeviceType`, `device_family`, `M4`, `amount_band`) use training-fitted string indexers serialized with the model.

4.7 Missing-value and class-imbalance strategies

Numeric medians and categorical missing policies are persisted. Serving uses numeric sentinels compatible with tree models.

Primary training uses class weights across all **412,956** rows. A secondary set undersamples legitimate cases:

Table 4.4: *Training and evaluation class distributions*

Dataset	Legitimate	Fraud	Ratio
`train_original`	**398,434**	**14,522**	**27.44**
`train_weighted`	**398,434**	**14,522**	**27.44**
`train_balanced`	**43,872**	**14,522**	**3.02**
`validation`	**85,450**	**3,036**	**28.15**
`holdout`	**85,993**	**3,105**	**27.70**

Validation and holdout prevalence remains untouched to preserve precision and calibration evaluation integrity.

4.8 Output contract

The pipeline exports curated Parquet splits: `train_original`, `train_weighted`, `train_balanced`, `validation`, `holdout`, and `kaggle_test`.

4.9 Verification result

The automated verifier passes **94** validation checks with zero failures, confirming dataset integrity, schema hashes, and split isolation, returning status:

```
ready_for_downstream_training
```

An older persisted manifest/snapshot used `verification_pending`; it is treated as historical evidence and must not be mixed with the official-run report when generating final metrics.

4.10 Data limitations

- Feature semantics are partly anonymized.
- Entity aggregates are batch features; a production online path is not implemented.
- The training matrix is collected to pandas before estimator fitting.
- There is one chronological validation window rather than temporal cross-validation.
- Native Windows is not the canonical full Parquet execution path.
- The local processed tree is large and intentionally excluded from Git.

CHAPTER 5

MODEL DEVELOPMENT, EVALUATION, AND SERVING

5.1 Artifact identity and terminology

This report separates four states that are otherwise easy to conflate. The historical V1/V2 comparison refers to earlier LightGBM artifacts and reports. The configured V2 software default is the version selected by serving configuration, not a promotion approval. The current official candidate is a CatBoost balanced package whose metadata also uses the version label `v2`; it is therefore called the *current CatBoost candidate* in this chapter. Its promotion decision is `not_promoted`, so this report does not claim that the candidate replaced the configured serving default.

Table 5.1: *Model identifiers and runtime roles*

Identifier	Model	Role	Status
Historical V2	LightGBM	Historical artifact	Historical
Serving default `v2`	LightGBM	Configured champion	Unchanged
Current candidate	CatBoost balanced	Challenger	Not promoted

5.2 Model-development protocol

The enhanced candidate workflow consumes the Spark/Parquet handover and executes the following ordered stages:

1. validate the data contract and canonical feature order;
2. fit categorical mappings on training data only;
3. train a Logistic Regression baseline;
4. compare Logistic Regression, LightGBM, XGBoost, and CatBoost, including balanced variants;
5. select and tune a candidate using the selection window;
6. fit an isotonic calibrator on the calibration window;
7. choose review and reject thresholds on the policy window;
8. freeze estimator, mappings, calibrator, and policy metadata;
9. evaluate the frozen package on holdout;
10. write candidate metadata, checksums, smoke evidence, and a promotion decision.

The feature loader removes `TransactionID`, `isFraud`, `class_weight`, `split_name`, processing metadata, and `generated_at` from the model vector. The canonical vector contains 68 features, 61 numeric and 7 categorical, and includes `TransactionDT`. This prevents a pre-

vious contract failure in which the string `train_weighted` entered the numeric training frame.

The boundary must be stated precisely as: **distributed data preparation with single-node Python model training**. Spark writes Parquet; Python training converts model matrices to pandas. There is no evidence that LightGBM, XGBoost, or CatBoost is distributed over Spark workers. The end-to-end model development lifecycle, spanning validation, hyperparameter tuning, calibration, and policy selection, is depicted in Figure 5.1.

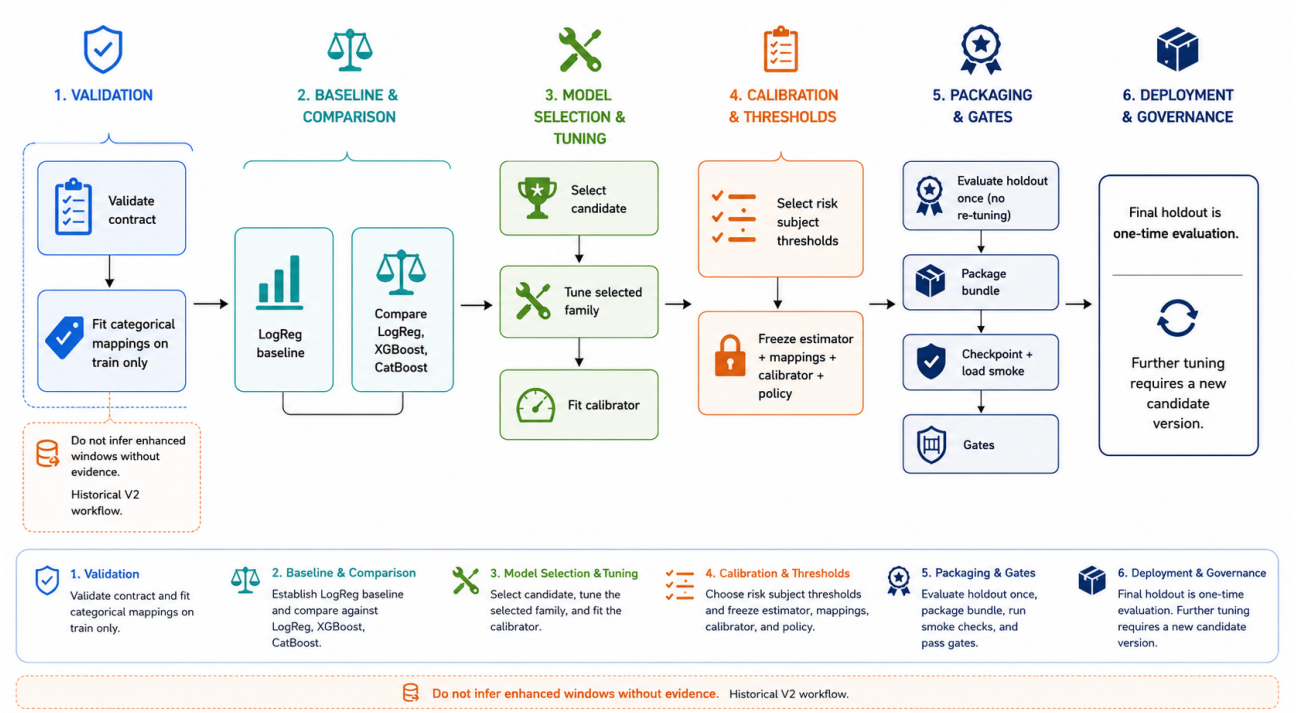


Figure 5.1: *End-to-end model training lifecycle. Illustrates model selection, tuning, fitting the calibrator within the Calibration & Thresholds stage, and holdout evaluation.*

5.3 PySpark MLlib DecisionTree baseline

To satisfy BDA501 big-data machine-learning baseline requirements, the PySpark preprocessing module implements a PySpark MLlib `DecisionTreeClassifier` pipeline. The classifier evaluates the canonical **68**-feature vector across three dataset variants (`baseline_tree`, `weighted_tree`, and `undersampled_tree`) with hyperparameters `maxDepth=8`, `maxBins=128`, and `minInstancesPerNode=50` [6].

Table 5.2 records the reproducible metrics saved in the evaluation metrics report. This PySpark MLlib pipeline serves as a data-quality baseline and must not be confused with single-node Python gradient-boosting candidate comparisons (CatBoost, LightGBM, XGBoost).

Table 5.2: *PySpark MLlib DecisionTree Baseline Performance*

Model variant	Split	PR-AUC	ROC-AUC	Precision	Recall	Time (s)
baseline_tree	Validation	**0.0205**	**0.2329**	**0.6928**	**0.1574**	**13.5**
baseline_tree	Holdout	**0.0209**	**0.2366**	**0.6641**	**0.1948**	**13.5**
weighted_tree	Validation	**0.2666**	**0.6866**	**0.1564**	**0.6400**	**12.1**
weighted_tree	Holdout	**0.2773**	**0.6984**	**0.1494**	**0.6583**	**12.1**
undersampled_tree	Validation	**0.0284**	**0.4265**	**0.2791**	**0.5217**	**5.7**
undersampled_tree	Holdout	**0.0278**	**0.4163**	**0.2039**	**0.5076**	**5.7**

5.4 Candidate models and selection

The baseline is Logistic Regression. The compared gradient-boosting families are LightGBM, XGBoost, and CatBoost. The current CatBoost candidate comparison selects `catboost__balanced`; the candidate manifest identifies the final model as CatBoost trained on the balanced variant.

Table 5.3: *Current CatBoost candidate training evidence*

Field	Value
Artifact version label	`v2`
Selected model	CatBoost
Training variant	`balanced`
Feature count	**68**
Selection window	**44,243** rows
Calibration window	**22,122** rows
Policy window	**22,121** rows
Calibration method	Isotonic
Processing version	`ieee-cis-preprocess-2.1.0`
Feature schema	`ieee-cis-features-1.1.0`

The saved comparison artifact records model-family and balanced-variant results. PR-AUC is the primary selection metric due to class imbalance; ROC-AUC is retained as a complementary ranking metric.

Table 5.4: *Current model-family selection evidence*

Model	Training variant	ROC-AUC	PR-AUC	Decision
Logistic Regression	Weighted	**0.8219**	**0.3090**	Baseline
LightGBM	Weighted	**0.9116**	**0.5682**	Compared
XGBoost	Weighted	**0.8879**	**0.5529**	Compared
CatBoost	Weighted	**0.8757**	**0.5245**	Compared
CatBoost	Balanced	**0.9070**	**0.5703**	Selected

CatBoost Balanced was selected for achieving the highest selection-window PR-AUC.

5.5 Temporal development windows

The validation period is partitioned into selection, calibration, and policy windows. Model family and parameters are selected in the selection window, isotonic calibration fits in the calibration window, and decision thresholds fit in the policy window. Holdout remains isolated until final evaluation.

5.6 Current CatBoost candidate results

Table 5.5: *Current CatBoost candidate validation and holdout metrics*

Window	Threshold	PR-AUC	ROC-AUC	Precision	Recall	Brier
Validation selection	**0.50**	**0.5806**	**0.9081**	**0.3970**	**0.6220**	–
Holdout	**0.16**	**0.4266**	**0.8781**	**0.3820**	**0.5182**	**0.0248**

Figures 7.3a–7.4 summarize the candidate's holdout performance.

5.7 Calibration and threshold policy

The candidate uses isotonic probability calibration with threshold specifications:

Table 5.6: *Current candidate policy artifact*

Policy field	Value
Review threshold	**0.16**
Reject threshold	**0.53**
Minimum review precision	**0.30**
Minimum reject precision	**0.60**
Selected review precision	**0.3371**
Selected reject precision	**0.7398**

The backend Risk Scoring module maps probabilities to a **0–100** score, using fixed runtime bands:

Table 5.7: *Candidate policy versus backend runtime policy*

Decision	Candidate policy	Backend runtime policy	Active authority
Review starts	**0.16**	**0.40**	Backend
Reject starts	**0.53**	**0.80**	Backend

5.8 Packaging, checksum, and promotion

The candidate bundle packages joblib model weights, training metadata, calibrator, threshold config, metrics, manifest, promotion decision, and SHA-256 checksums.

Table 5.8: *Current candidate promotion gate*

Gate	Result	Evidence
Holdout PR-AUC ≥ 0.4582	Fail	0.4266435
Holdout ROC-AUC ≥ 0.8746	Pass	0.8780755
Review precision ≥ 0.30	Pass	0.3820038
Calibration non-regression	Pass	Brier decreased
Artifact checksum	Pass	SHA-256 match
Artifact-load smoke	Pass	recorded in manifest
Data contract verification	Pass	ready, 94 checks
Version match	Pass	`v2`
Git clean	Fail	`git_dirty=true`

The resulting decision is `not_promoted`, with `serving_version_unchanged=true`. The current candidate therefore did not replace the configured V2 software default. This is a governance outcome, not a claim that the candidate became V2 or that the historical LightGBM V2 artifact is identical to the current CatBoost candidate.

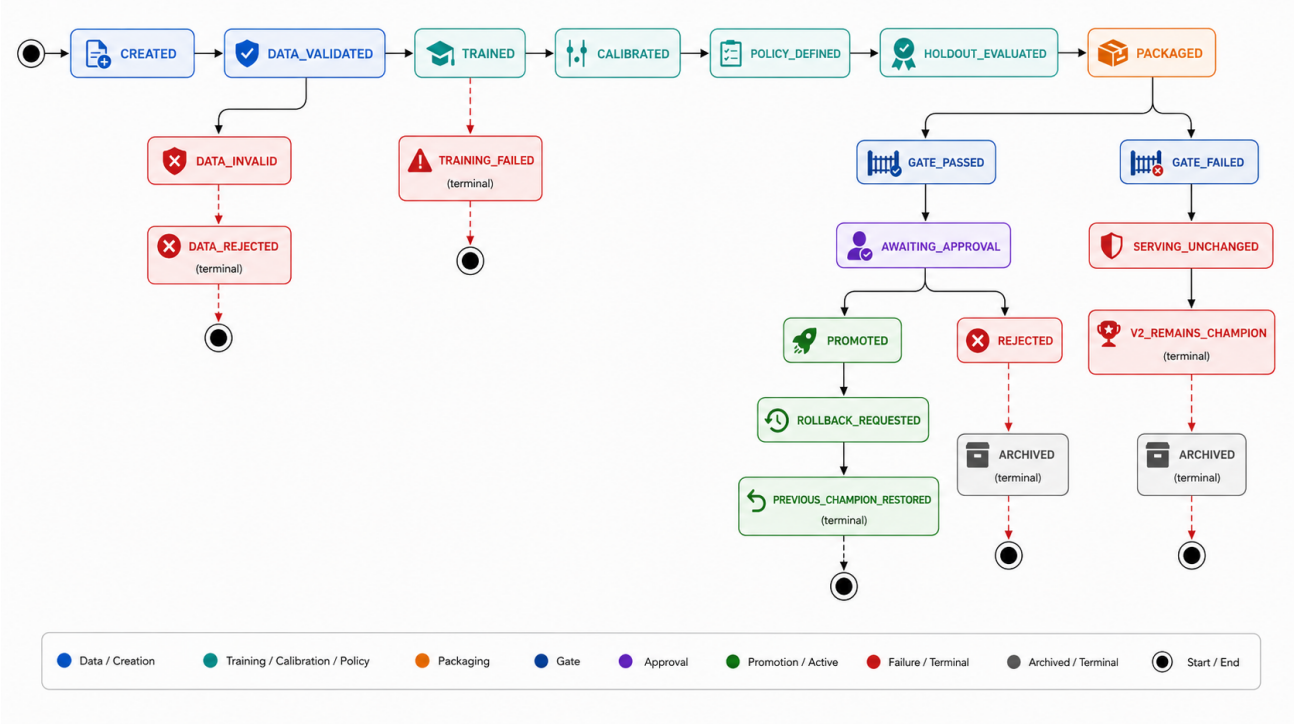


Figure 5.2: *Promotion state machine. A failed candidate enters a failure/archive path while serving configuration remains unchanged.*

5.9 Request-time scoring and explanations

The scorer loads one artifact into a process cache and maps raw categorical strings using training-derived dictionaries. It aligns columns to the canonical order, converts missing values to the configured sentinel, calls `predict_proba`, applies an available calibrator, and returns the top five absolute TreeSHAP contributions for supported tree models. SHAP is a local model attribution and not a causal explanation. When calibration changes the displayed probability,

the current SHAP values still describe the raw model output unless a separate calibrated explanation method is used.

The response distinguishes `full_feature` from `partial_demo`. Full-feature scoring requires the canonical engineered vector. Partial-demo scoring defaults missing fields and is useful for UI continuity, but it cannot recreate card, email, and device histories from a small request. It must not be described as real-time full-feature scoring.

5.10 Serving configuration and MLflow

Configuration defaults `FRAUD_MODEL_SERVING_VERSION=v2`, while the current candidate decision remains `not_promoted`. The safe wording is "v2 configured software default", not "production-promoted model". Setting `FRAUD_MODEL_SERVING_VERSION=v1` remains the rollback/configuration path, subject to artifact compatibility.

MLflow is implemented as local experiment tracking and lineage using SQLite and local artifact storage. The repository does not evidence a central Model Registry, registered model versions, stage transitions, or an automatic promotion authority. Candidate packaging and promotion JSON are evidence files, not a serving registry.

5.11 Model-readiness decision

The current CatBoost candidate is suitable for continued academic evaluation and manual review of its evidence. It should not be described as a production model until a frozen application smoke loads the expected artifact without fallback, the runtime policy is explicitly approved and aligned with the candidate, a durable registry/checksum handover exists, and online historical feature parity is addressed.

CHAPTER 6

APPLICATION AND DEPLOYMENT

6.1 Integrated serving architecture

The application is an **in-process model-serving modular monolith**. FastAPI imports the scoring package directly, persists scored transactions and review outcomes in PostgreSQL or SQLite, and exposes them to a React dashboard. This removes a network model-service boundary and is suitable for the demonstration. The tradeoff is coupling between API startup, model dependencies, image size, and scaling.

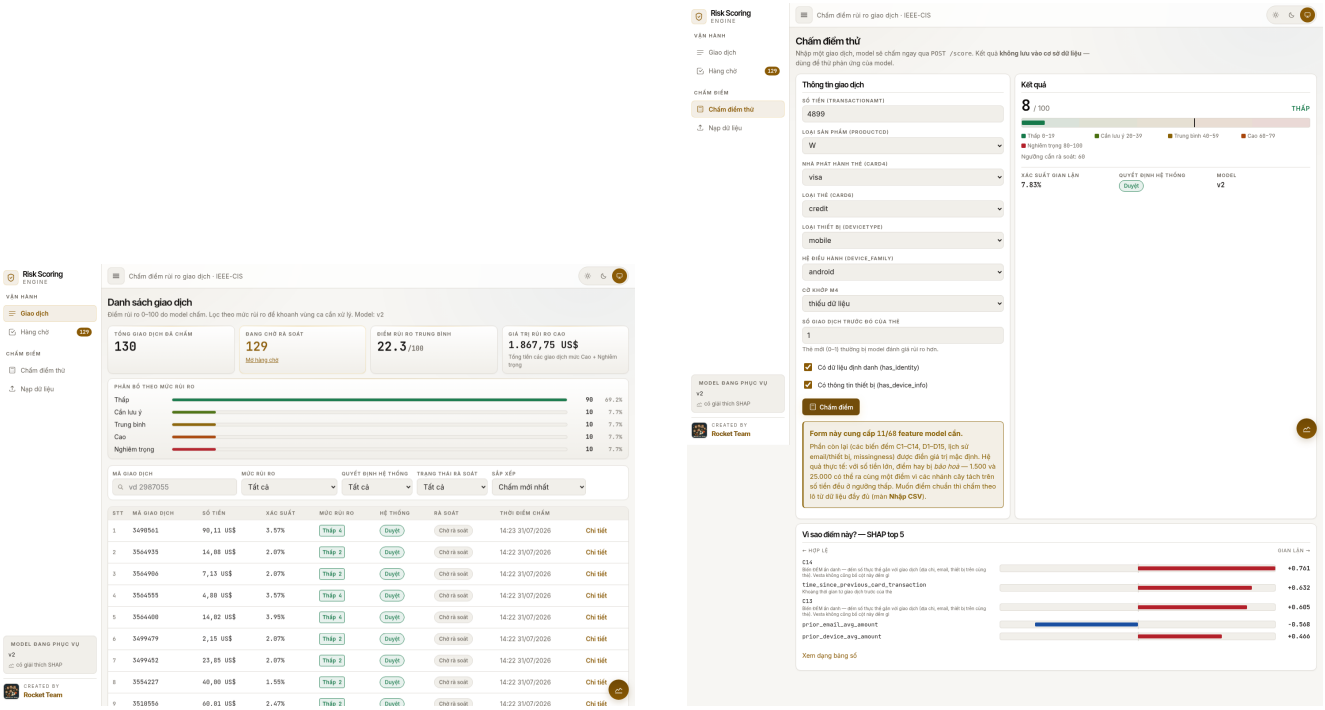
6.2 Scoring and review workflow

The backend validates requests, loads the configured scoring artifact, returns probability, risk score, fixed risk band, automatic decision, model metadata, and optional TreeSHAP values. Transactions retain the raw feature payload, score, timestamp, and model version. A separate review record stores reviewer status, fraud/legitimate label, and notes, so automatic decisions remain distinct from human outcomes.

The HTTP contract covers scoring, transaction exploration, CSV import, processed dataset loading, background jobs, and review. Endpoint-level paths and response fields are preserved in Appendix A and in the repository API contract. The main architecture issue is feature availability: full-feature scoring requires the canonical engineered vector, while partial-demo scoring defaults missing fields. The latter cannot recreate card, email, or device history from a small request and is not production-equivalent scoring.

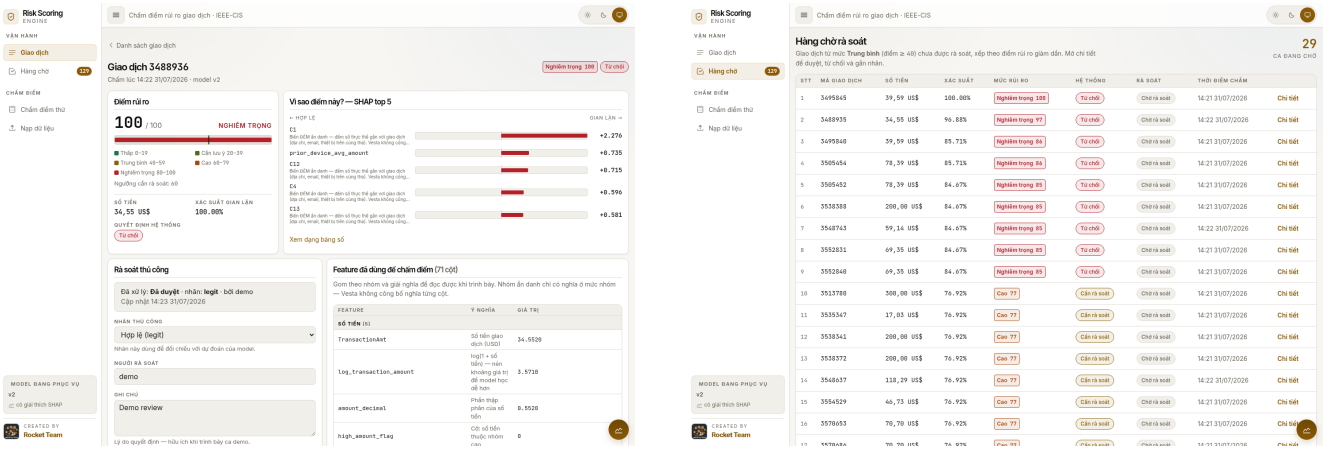
6.3 Frontend integration

The React interface presents transaction lists, score details, risk bands, SHAP values, review actions, imports, and partial scoring. Mock responses can support UI development, but they are not model evidence. The interface uses text and numeric alternatives to colour and displays an explicit unavailable state when SHAP is absent. Figure 6.1 and Figure 6.2 display the operational user interface components of the fraud detection system.



(a) *Transaction monitoring queue.* **(b)** *Risk score and SHAP feature importance.*

Figure 6.1: *Application User Interface: Transaction queue and real-time risk scoring breakdown.*



(a) *Transaction detail panel.* **(b)** *Analyst fraud review workflow.*

Figure 6.2: *Application User Interface: Transaction inspection and analyst review interface.*

6.4 Deployment boundary

Docker Compose separates preprocessing, training, and application profiles. The backend receives processed Parquet through a read-only mount and installs the model package as a path dependency. Spark workers support distributed data preparation; estimator fitting remains single-node after the Parquet-to-pandas boundary. The deployment diagram is retained in Appendix B because it is implementation-level detail rather than a main architecture decision.

6.5 Operational limitations

The application currently has fail-open heuristic fallback, in-memory job state, synchronous CSV import, coupled backend/model dependencies, no authentication or audit event log, and

no automated browser test suite. These limitations are consolidated with data and governance gaps in Chapter 7; they are not repeated as separate readiness claims throughout the thesis.

CHAPTER 7

VALIDATION AND WHOLE-SYSTEM REVIEW

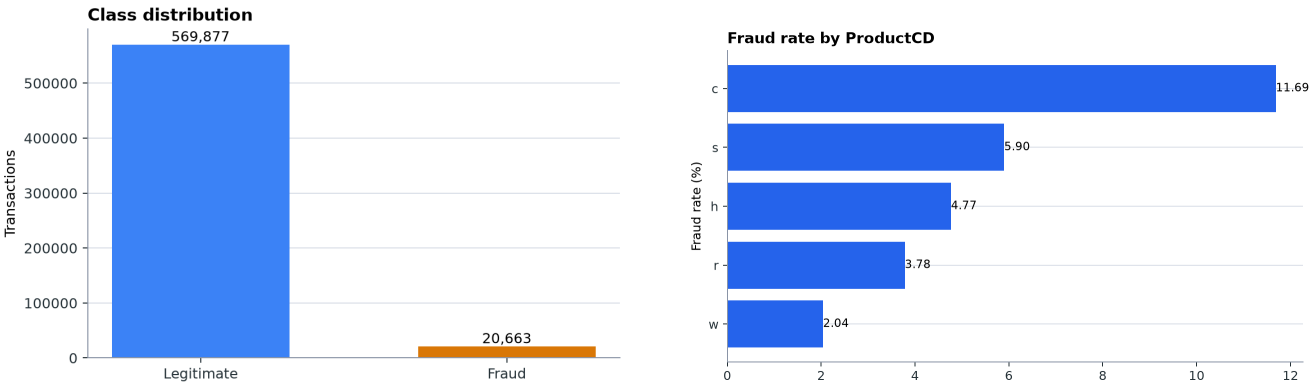
7.1 Evidence status

Claims in this thesis are classified as runtime-verified, artifact-reported, implemented from source, or proposed. The current official processed-data report records **94** verification checks passed and zero failed, with status `ready_for_downstream_training`. An older manifest/facts snapshot uses `verification_pending`; it is retained as historical evidence and is not combined with the official-run metrics.

7.2 Data and result evidence

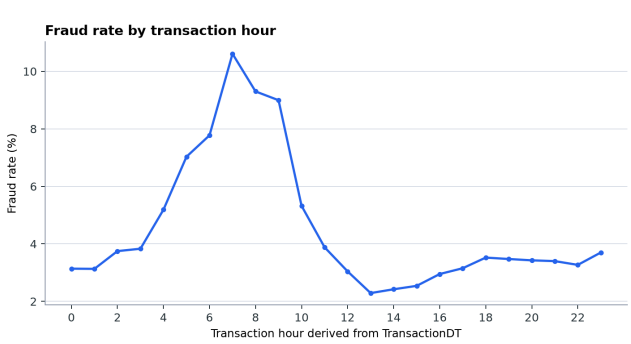
The verifier checks dataset presence, row counts, identifiers, labels, class weights, feature order, schema hashes, chronological order, split overlap, and required artifacts. The official candidate metadata reports **68** features, the selection/calibration/policy windows, isotonic calibration, holdout metrics, checksums, and a load-smoke result. These artifacts support the data-contract and candidate claims; they do not prove online feature parity or deployment readiness.

The following charts are descriptive evidence from the saved EDA and current candidate artifacts. They show class imbalance, selected group rates, model family selection, validation-to-holdout degradation, confusion counts, and calibration. None should be read as causal analysis.

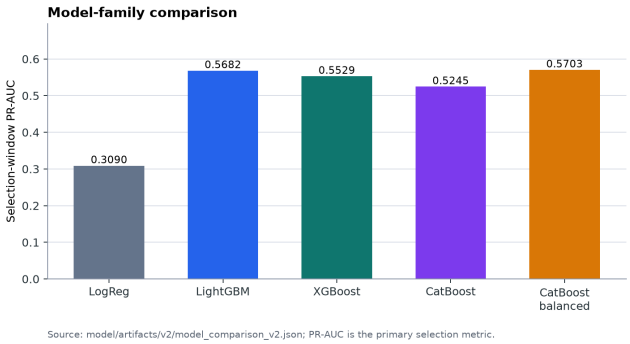


(a) *Class distribution.* **(b)** *Fraud rate by ProductCD.*

Figure 7.1: *Exploratory Data Analysis: Class distribution and fraud rate by ProductCD.*

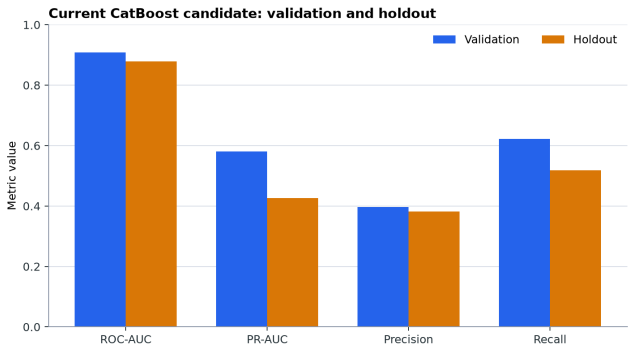


(a) *Fraud rate by transaction hour.*

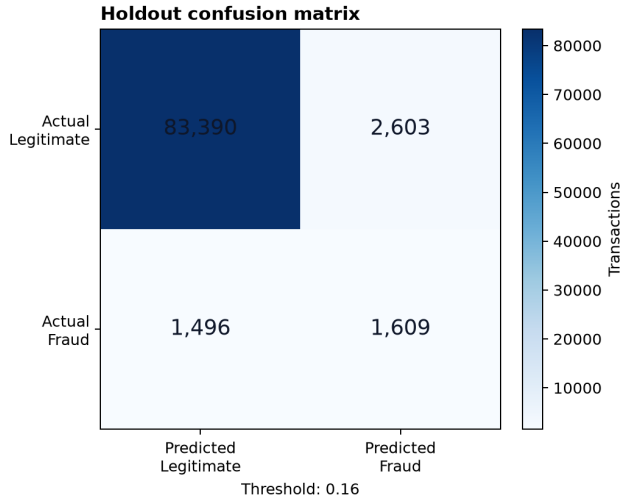


(b) *Model-family selection PR-AUC.*

Figure 7.2: *Temporal fraud association and model-family performance selection.*



(a) *Validation vs holdout metrics.*



(b) *Holdout confusion matrix (0.16 threshold).*

Figure 7.3: *Candidate holdout evaluation: Metrics drop and operating confusion matrix.*

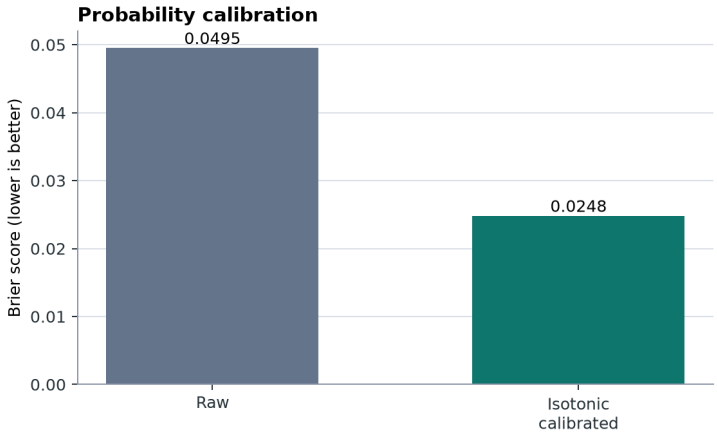


Figure 7.4: *Probability calibration: Raw vs isotonic-calibrated Brier score on holdout.*

7.3 Technical challenges and implemented solutions

Table 7.1 summarizes the major engineering and data challenges encountered during system development, alongside their implemented solutions and repository evidence.

Table 7.1: *Technical Challenges, Implemented Solutions, and Evidence*

Challenge	Implemented solution	Evidence
Transaction–identity sparsity	Transaction-authoritative LEFT JOIN preserving 100% transaction rows with missingness indicators	Preprocessing Join Audit Step
Missing values	Train-only median imputation for numeric; explicit `'missing'` category	Feature Contract Specification
Class imbalance	Evaluated `original`, `weighted` (weight 3.0), and `balanced` undersampled splits	Balanced Parquet Artifact
Temporal leakage	Chronological split on `TransactionDT` (`train` → `val` → `calib` → `holdout`)	Leakage-free Pipeline Stages
Schema & feature order	Frozen canonical 68-feature vector and machine-checked schema hash gate	Verification Engine Gate
Verifier failure	Automated validation gate outputting `ready_for_downstream_training`	Verification Report Artifact
Spark-to-pandas boundary	Spark handles distributed data preparation; Python fits single-node matrices	Model Execution Logs
Offline–online feature mismatch	Standardized backend feature loader mapping scoring request vector to schema	Backend Feature Loader

7.4 Integrated validation findings

The strongest verified boundary is the Spark-to-Parquet handover: schemas, split membership, and learned preprocessing artifacts are available for downstream training. The strongest model evidence is the current CatBoost balanced candidate's artifact-reported validation and holdout metrics. The strongest governance evidence is the explicit `not_promoted` decision and `serving_version_unchanged=true`; a failed candidate did not alter the configured V2 software default.

The application evidence is narrower. Source and subsystem tests cover model feature alignment, unseen categories, missing values, backend risk bands, and selected API behaviours. The stored full-stack execution history includes dependency and build remediation, but it is not a complete production smoke. Frontend lint/build evidence and model tests should therefore be described as subsystem evidence, not as proof of an end-to-end operational service.

7.5 Consolidated limitations

The review consolidates limitations into five decision-relevant groups:

1. **Reproducibility:** local MLflow tracking, workspace-local artifacts, snapshot discrepancies, and single-node pandas training limit portability and scale.
2. **Policy alignment:** the candidate threshold artifact and calibration metadata are not the backend's fixed risk-band authority.
3. **Offline–online parity:** batch historical aggregates are not generated by an online feature service; partial-demo defaults are not equivalent to full-feature scoring.
4. **Operational controls:** fallback is fail-open for demonstration continuity, jobs are in memory,

and monitoring, migrations, and bulk re-scoring are absent.

5. **Security and governance:** authentication, authorization, durable audit, immutable registry authority, and production drift/latency monitoring are not evidenced.

These gaps define the boundary of the contribution. They do not invalidate the academic data and model workflow, but they prevent a production-readiness claim.

7.6 Recommended validation sequence

Before any future serving decision, reproduce the official candidate from a clean dependency and data snapshot, load the exact checksum without fallback, align the runtime policy with the approved calibration/threshold evidence, and test full-feature scoring with historical features generated by the same contract. Only then should an independent candidate version be compared and a human approval recorded.

CHAPTER 8

CONCLUSION AND FUTURE WORK

8.1 Conclusion

The project demonstrates an end-to-end fraud scoring system separating distributed Spark data preparation from single-node Python model training. CatBoost Balanced achieved validation ROC-AUC **0.9081** and PR-AUC **0.5806** (holdout ROC-AUC **0.8781**, PR-AUC **0.4266**), while candidate status remains `not_promoted`.

8.2 Summary of research question answers

- **RQ1 (Data Integrity)**: Leakage-free chronological splits and schema contracts are verified.
- **RQ2 (Classifier Performance)**: CatBoost achieved highest PR-AUC; performance degrades gracefully on holdout.
- **RQ3 (Serving Architecture)**: In-process Python model package enables Spark-free real-time scoring.
- **RQ4 (System Boundaries)**: Architectural limits (offline-online parity, fail-open fallback) bound the demo.

8.3 Future work

1. Implement an online feature store for real-time aggregation parity.
2. Separate runtime policy configuration from offline model weights.
3. Integrate production audit logging, security, and automated drift monitoring.

BIBLIOGRAPHY

[1] IEEE Computational Intelligence Society, “Ieee-cis fraud detection,” 2019.

[2] Amazon Science, “Fraud dataset benchmark: Ieee-cis fraud detection,” 2023.

[3] J. Davis and M. Goadrich, “The relationship between precision-recall and roc curves,” in *Proceedings of the 23rd International Conference on Machine Learning*, pp. 233–240, 2006.

[4] T. Saito and M. Rehmsmeier, “The precision-recall plot is more informative than the roc plot when evaluating binary classifiers on imbalanced datasets,” *PLOS ONE*, vol. 10, no. 3, p. e0118432, 2015.

[5] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[6] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. Tsai, M. Amde, S. Owen, R. Xin, R. Xin, M. J. Franklin, R. R. Zadeh, M. Zaharia, and A. Talwalkar, “Mllib: Machine learning in apache spark,” *Journal of Machine Learning Research*, vol. 17, no. 34, pp. 1–7, 2016.

CHAPTER A

API REFERENCE

A.1 Common enums

```
RiskBand = low | guarded | medium | high | critical
Decision = approve | review | reject
ReviewState = pending | approved | rejected
ReviewLabel = fraud | legit
ScoringMode = full_feature | partial_demo
```

A.2 Core response fields

Table A.1: *Core API fields*

Field	Type	Meaning
`fraud_probability`	float	Model/fallback output in $[0, 1]$
`risk_score`	integer	Rounded probability times 100
`risk_band`	enum	Fixed backend score band
`decision`	enum	Automatic backend policy result
`scoring_mode`	enum	Full vector or partial demonstration
`shap_top5`	list/null	Local model contributions when available
`model_version`	string	Artifact version used for the score
`processing_version`	string/null	Data processing lineage
`feature_schema_version`	string/null	Feature contract lineage

A.3 Endpoint summary

Method	Path	Response
GET	`/health`	Process status
GET	`/meta`	Model and feature metadata
POST	`/score`	Score response
GET	`/transactions`	Paginated transactions
GET	`/transactions/stats`	Aggregate statistics
GET	`/transactions/import/template`	UTF-8 CSV
POST	`/transactions/import`	Import outcome and coverage
GET	`/transactions/{id}`	Transaction detail
POST	`/transactions/{id}/review`	Updated detail
GET	`/datasets`	Available datasets

Method	Path	Response
POST	`/data/load`	Background job
GET	`/jobs/{id}`	Job state
GET	`/jobs/active/current`	Job or null
POST	`/jobs/{id}/cancel`	Updated job

The normative, implementation-aligned contract is maintained in the repository API contract specification.

CHAPTER B

DEPLOYMENT AND IMPLEMENTATION DIAGRAMS

B.1 Application deployment

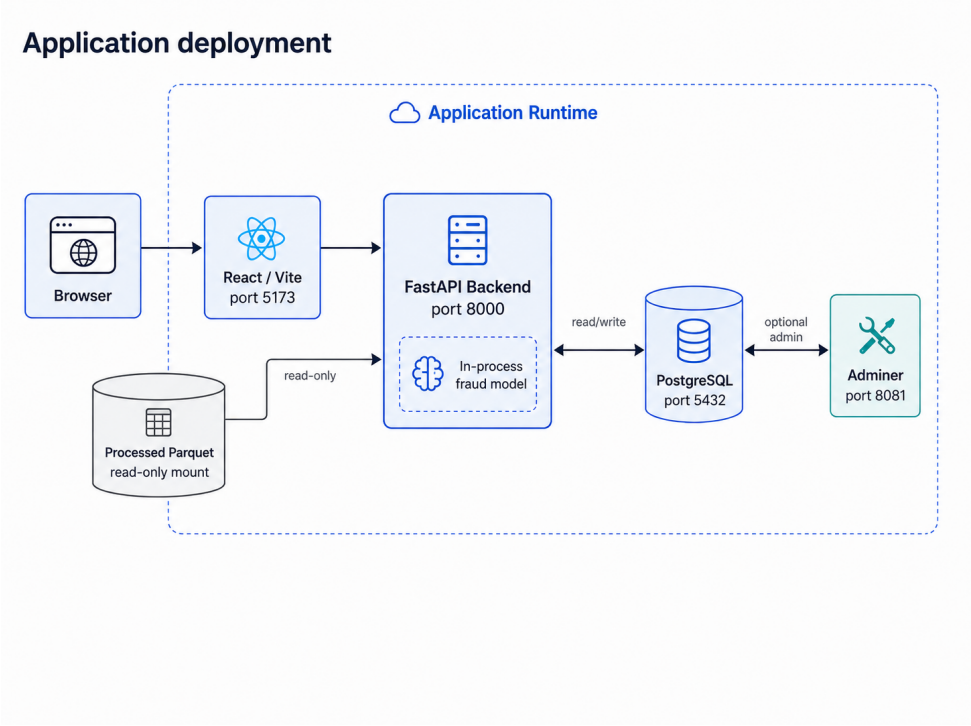


Figure B.1: *Default application deployment. Browser, React/Vite, FastAPI backend, PostgreSQL, and in-process fraud model.*

The application diagram is implementation-level evidence for the modular monolith. Spark preprocessing and Python model training are separate profiles; the diagram does not imply distributed estimator fitting.

CHAPTER C

FEATURE AND RISK CONTRACTS

C.1 Canonical feature groups

Table C.1: *Canonical feature groups*

Group	Representative features
Amount	`TransactionAmt`, log/capped/decimal amount, amount band, card-mean ratios and deviations
Time	`TransactionDT`, day, week, hour, day-of-week proxy, age, night flag
Missingness	Selected/identity missing counts and ratios
Presence	Identity, device, payer/recipient email, distance, address, same-domain flags
Anonymous behavior	`dist1`, `dist2`, selected `C1--C14`, selected `D1--D15`
Card history	Prior count/sum/mean/stddev and time since previous transaction
Email history	Prior count/sum/mean
Device history	Prior count/sum/mean
Categorical	`ProductCD`, `card4`, `card6`, `DeviceType`, `device_family`, `M4`, `amount_band`

C.2 Forbidden model columns

```
TransactionID
isFraud
class_weight
split_name
processing_version
feature_schema_version
generated_at
```

C.3 Risk bands

Minimum	Maximum	Band	Decision
0	19	low	approve
20	39	guarded	approve
40	59	medium	review
60	79	high	review
80	100	critical	reject

The risk bands are a fixed demonstration policy. They are not derived from the saved V2 validation operating point.

CHAPTER D

REPRODUCIBLE RUNBOOK

D.1 Raw data placement

```
data/data/ieee-fraud-detection/
  train_transaction.csv
  train_identity.csv
  test_transaction.csv
  test_identity.csv
  sample_submission.csv # optional
```

D.2 Preprocessing

```
docker compose -f docker-compose.preprocessing.yml build
docker compose -f docker-compose.preprocessing.yml run --rm preprocess
docker compose -f docker-compose.preprocessing.yml run --rm verify-
   processed
```

If a completed Spark export lacks a finalized manifest:

```
python -m pipeline.processed_contract \
  --output-dir data/processed/ieee_cis_fraud_risk
python -m pipeline.verify_processed_data \
  --output-dir data/processed/ieee_cis_fraud_risk
```

D.3 Model training

```
bash scripts/run_official_full.sh
```

The official workflow runs preprocessing, contract validation, baseline, comparison, tuning, calibration, threshold analysis, holdout evaluation, packaging, checksum generation, and the promotion gate. The current official decision is `not_promoted`; the serving configuration remains unchanged. Holdout evaluation must occur only after model and policy selection are fixed.

D.4 Application

```
docker compose up --build
```

Then open:

- dashboard: `http://localhost:5173`;
- API documentation: `http://localhost:8000/docs`;

- optional Adminer: `http://localhost:8081`; select the `PostgreSQL` system, then use the configured database credentials.

D.5 Local development

```
# Terminal 1
cd backend
uv sync --frozen
uv run uvicorn fraud_backend.main:app --reload

# Terminal 2
cd frontend
npm ci
npm run dev
```

D.6 Rollback

```
FRAUD_MODEL_SERVING_VERSION=v1
```

Restart the backend and verify `/meta` before accepting traffic. The rollback setting is a configuration path, not evidence that V1 has been validated as the current champion in the official snapshot.

CHAPTER E

TEST AND CONTRIBUTION MATRIX

E.1 Recommended validation commands

```
# Processed data (read-only unless --write-report is supplied)
python -m pipeline.verify_processed_data \
  --output-dir data/processed/ieee_cis_fraud_risk

# Model
cd model
uv run ruff check .
uv run pytest -q

# Backend
cd backend
uv run ruff check src tests
uv run pytest -q

# Frontend
cd frontend
npm run lint
npm run build
python scripts/check-ui.py
```

E.2 Acceptance scenarios

Table E.1: *Acceptance scenarios*

Scenario	Expected evidence
Processed handoff	Verifier passes every schema, row, split, and artifact check
Real model startup	`/meta` returns expected version with no warning
Full-feature score	`scoring_mode=full_feature`; probability bounded
Partial score	`partial_demo` is visible; required fields enforced
Unseen category	Request succeeds with training-defined unknown mapping
SHAP absent	UI renders explicit unavailable state
Review update	Status/label persist and queue/statistics invalidate

Scenario	Expected evidence
CSV row error	Good rows commit; bad row appears in bounded error list
Job reconnect	Active job is returned after browser refresh
Model rollback	V1 loads and `/meta` reports V1

E.3 Team roster and repository role mapping

Table E.2: *Group 1 roster and recorded subsystem contributions*

Team member	Repository evidence
Dương Bình An	Data processing, EDA, feature engineering, contracts, and handover
Lê Quang Tuyến	Named team member; a subsystem-specific role is not stated in the reviewed repository
Phạm Đức Long	React dashboard, risk/SHAP presentation, review UX, and import UI
Đỗ Quốc Trung	FastAPI, SQL persistence, imports, background data loading, and Docker
Dương Hồng Quân	Model training, comparison, SHAP, artifact packaging, and scoring
Nguyễn Lê Hồng Nhi	Named team member; a subsystem-specific role is not stated in the reviewed repository

The full roster and supervisor were supplied by the project team. Detailed contribution statements should be confirmed by all members before submission.